

PAHS 学习全集

****Zachary Zhou 专用 — 架构、流程、代码、Co-op 保底、大哥汇报****

日期 -> 2026 年 6 月
项目 -> PAHS `~/Desktop/PAHS`
配套 -> SMAS `~/Desktop/SMAS`, PIP `~/Desktop/PIP`
阶段 -> Gap year, 2026 年 9 月 SFU CS 入学, Co-op 准备

如何使用本 PDF

- 1. 第一遍：读 Part 1-3 (为什么做、大哥理论、生态)
- 2. 第二遍：读 Part 4-7 (架构、流程、该学的代码)
- 3. 第三遍：用 Part 10 自测题，合上电脑口头回答
- 4. 每周：对照 Part 11 学习单元 + 用 Part 12 模板向大哥汇报

Part 1 · 你的处境与学习目标

1.1 你是谁

- * 20 岁, Burnaby, Gap year
- * 2026 年 9 月 SFU Computer Science 入学
- * 已有项目：PAHS (主控)、SMAS (IG 图文)、PIP (短视频)、EduSync、Popboxx 相关
- * 工具：Cursor + DeepSeek API, 暂不用本地大模型

1.2 两条线，不打架

[竖杠 (Co-op 保底)] 课业、面试、能读能改代码 (合格：核心文件、Python、Git、LeetCode、系统设计基础)

这叫 T 型能力：有主轴能落地，有横杠能判断价值。

1.3 本项目你要学到什么程度

合格标准 (能口头回答即可)：

- 1. 用户一句话后，经过哪几步才出结果？
- 2. PAHS、SMAS、PIP 各自干什么？
- 3. Path A 和 Path B 区别？
- 4. 坏了先查哪一层？
- 5. 哪些能自动，哪些必须人审？
- 6. 能指出 6-8 个核心代码文件各自职责

不必：从零重写整个 PAHS；背每一行 AI 生成的代码。

Part 2 · 大哥 (Sam) 那句话怎么理解

2.1 原话三层

「真正要学的是能力能做多少；不需要学代码怎么写；因为已经没用了。」

- 方法 -> 别用旧办法死背语法当唯一能力
判断 -> 手写语法的稀缺性在下降，但判断力不会

2.2 他站在什么位置

项目负责人视角：能不能交付、能不能规模化、能不能帮 Popboxx/餐饮等场景干活。

他投资的是：真实痛点到可审核交付这条链，不是你会不会背 Python。

2.3 「能力能做多少」四个问法

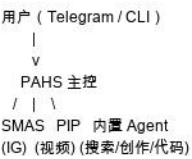
- 1. 范围 Scope：能从一句话做到预览图吗？能发帖吗？差在哪？
- 2. 稳定 Reliability：十次里几次成？挂在哪？
- 3. 速度 Speed：从需求到可给客户看要多久？
- 4. 边界 Boundary：什么绝不该承诺（开账号、付款、代发帖）？

2.4 映射到生活（简表）

PAHS -> 这周又多交付一件事，还是又多一个半成品？
餐厅工作 -> 今天多帮客人解决一件事了吗？
身体作息 -> 睡眠和运动是「能持续交付」的基础吗？

Part 3 · 项目生态

3.1 一张关系图（文字版）



3.2 各项目角色

[SMAS] IG 图文 + 预览图（合格：external_agents.yaml）
[PIP] 短视频生成（合格：external_agents.yaml）

PAHS 通过 subprocess 桥接调用 SMAS/PIP，不是把它们的代码拷进 PAHS。

3.3 两条执行路径

[过 Orchestrator] 是（合格：否）
[适合] 调研、多步、复杂任务（合格：快速出 IG/视频）
[人审] milestone + 总反馈（合格：SMAS：好 / 改：...）

学习时两条都要懂。产品体验靠 B；系统完整度和简历靠 A。

Part 4 · 架构分层

- [Orchestrator] 写 ExecutionPlan 任务表（合格：planning/orchestrator_planner.py）
- [Step Router] 计划结构校验（合格：planning/step_router.py）
- [Harness] 规则、预算、验证、能力边界（合格：harness/）
- [Agents] Searcher / Creator / Executor（合格：agents/）
- [Search Router] Tavily vs Perplexity（合格：agents/search_router.py）
- [External] SMAS / PIP 桥接（合格：external/）
- [Learner] 反馈到 rules / plan_patterns（合格：learning/）
- [Dev Lab] UI + 批量测试（合格：devlab/）

4.1 ExecutionPlan 结构

```
ExecutionPlan
intent_summary
complexity_band
review_policy
phases[]
phase: parallel?, depends_on
tasks[]: worker, tools, inputs_from
```

计划存 runs.plan_json，默认不展示给用户。预览：pah plan-preview "命令"

4.2 Harness 四层 + 能力边界

Tools -> 仅已批准工具

Validation -> verify_pipeline，失败可重试

Capability -> capability_brief：账号/发布/付款等诚实提示

Environment -> 预算、token、降级

Part 5 · LangGraph 完整流程 (Path A)

节点顺序 (src/pahs/graph/main.py)：

```
ingest
-> load_global_rules
-> triage_score
-> orchestrator_plan
-> plan_validate
-> env_precheck
-> load_target_rules
-> execute_plan_phase
-> verify_pipeline
-> present_milestone / final_present / retry / failed
-> milestone_human_review ( interrupt，等你 approved )
-> final_feedback_request ( interrupt，总反馈 )
-> Learner ( 总反馈后，提案需你 approve )
```

关键概念：

- * interrupt：图暂停，等你 pah reply 或 Dev Lab 里回复
- * checkpoint：SQLite 存状态，可 resume
- * BLOCKED：预算或环境检查不通过（批量测试已修每轮隔离预算）

Part 6 · 三套路由

【2】

序号：2

机制：Step Router

何时：plan 后

决定什么：计划是否合法

状态：结构校验

【3】

序号：3

机制：Search Router

何时：Searcher 执行

决定什么：Tavily vs Perplexity

状态：已完成

Search 两步：Step1 搜索 API 拿资料；Step2 DeepSeek 写最终报告。

Part 7 · 值得学的代码 (按优先级)

7.1 必读 (架构 + Co-op 能讲能改)

[P0] graph/runner.py (合格：start_run、resume_run)

[P0] planning/orchestrator_planner.py (合格：怎么生成计划)

[P0] planning/plan_executor.py (合格：phase 执行、并行、artifacts)

[P1] external/smas_bridge.py (合格：怎么调 SMAS)

[P1] external/pip_bridge.py (合格：怎么调 PIP、mock、超时)

[P1] harness/capability_brief.py (合格：能力缺口检测)

[P1] gateway/telegram_adapter.py (合格：Telegram 入口)

[P2] agents/searcher.py (合格：两步搜索)
[P2] agents/search_router.py (合格：搜索路由)
[P2] devlab/batch_runner.py (合格：批量测试怎么跑)
[P2] learning/learner.py (合格：提案怎么生成)
[P2] learning/batch_learner.py (合格：批量后整体分析)

7.2 配置必读

config/budget.yaml -> 预算限制
config/dev_batch_scenarios.yaml -> 批量测试 15 场景
.env -> API keys (勿提交 git)

7.3 Co-op 保底 (不在本 repo , 要并行)

- * 数据结构 + 算法 (LeetCode 中等)
- * Git、HTTP、REST、SQL 基础
- * 能读 Python、会 debug、会写小函数
- * 系统设计：能画「用户-API-DB」三层图

PAHS 是应用层实战；LeetCode + 课业是通行证。

Part 8 · Dev Lab 与批量测试

8.1 两种方式

[批量测试] pah dev-batch (合格：自动跑 N 次，出报告)

8.2 批量测试流程

dev_batch_scenarios.yaml
-> pah dev-batch --runs N --mock
-> 每轮 auto approve + synthetic feedback
-> dev_batch_report_*.md (成绩单)
-> dev_batch_improvement_plan_*.md (改进方案)
-> 你：Handoff 给 Cursor 改代码；approve 有用提案

8.3 Mock vs 真实 API

【真实】

模式：真实
命令：--no-mock
测什么：DeepSeek 文案与计划质量
建议次数：5-10

Mock 100/100 只说明流程稳，不代表 AI 质量合格。

8.4 清理

pah reset-test (输入 DELETE ALL) 清理 runs、提案、报告。

data/ 和 rules/learnings/*.json 已在 .gitignore。

Part 9 · 常用命令

pah pending / pah reply -> 审核
pah feedback "..." -> 总反馈
pah proposals pending -> 看 Learner 提案
pah proposals approve id -> 批准提案
pah dev-ui -> Dev Lab 网页
pah dev-batch --runs 100 --mock -> 批量 mock 测试

```
pah dev-batch --runs 10 --no-mock -> 真实 API 小批量
pah plan-preview "..." -> 预览计划
pah search-status -> 搜索配置
pah telegram -> 启动 bot
pah reset-test -> 清理测试数据
```

注意：python3 -m pabs.cli 或 venv 里 pah，不是 python3 pabs.cli

Part 10 · 自测题库 (合上电脑回答)

流程 6 问

1. Telegram 说「做 IG 图文」走哪条路径？经过哪些模块？
2. pah run 调研任务和直调 SMAS 有何不同？
3. milestone_review 和 final_feedback 区别？
4. Learner 提案为何不会自动生效？
5. capability_brief 检测到开账号任务时会怎样？
6. dev-batch 里 BLOCKED 曾经什么原因？现在怎么修的？

代码 4 问

1. runner.py 里 resume 做什么？
2. plan_executor 里 parallel 什么意思？
3. pip_bridge 在 dev-batch mock 时有什么不同？
4. batch_learner 和单次 learner 区别？

合格：每题能答 3 句话以上。

Part 11 · 四周学习单元

第 1 周：心智模型 + Path B

- * 读 Part 1-3、第五章 Telegram 走读 (详细版教程)
- * 动手：pah telegram，发一条 IG 需求
- * 自测：画出 Telegram 到 SMAS 的数据流

第 2 周：Path A + LangGraph

- * 精读 graph/main.py、runner.py
- * 动手：pah run + pah reply approved + feedback
- * 自测：列出 10 个节点名

第 3 周：Orchestrator + External

- * 精读 planning/、external/
- * 动手：pah plan-preview；改 external_agents.yaml 一个关键词
- * 自测：解释 ExecutionPlan 三字段

第 4 周：Harness + 测试 + 汇报

- * 精读 capability_brief、batch_runner、batch_learner
- * 动手：dev-batch --runs 10 --mock；读 improvement_plan
- * 自测：向 Sam 做 5 分钟 demo 讲解

并行：每周 3-5 道 LeetCode；SFU 预习课若有则跟进。

Part 12 · 向大哥 (Sam) 汇报模板

12.1 每周三段式 (微信/语音均可)

****本周交付 (结果)****

- * 做成了什么客户能感知的事 (例 : Telegram 能出 IG 预览 ; 批量测试 100 次全过)

****能力边界 (诚实)****

- * 现在能做到哪一步 ; 还做不到什么 (例 : 不能代发帖、不能开平台账号)

****下周一件事 (可验证)****

- * 只承诺一件具体交付 , 不要列 10 个大计划

12.2 他爱听的词汇

- * 稳定、可审核、可复用、省人力、开业引流、预览给客户看
- * 少讲 : 我学了很多代码、我改了 50 个文件

12.3 避免的汇报方式

- * 只讲过程不讲结果
- * 承诺全自动发帖/开账号 (系统故意不做)
- * 一堆技术名词没有 demo

Part 13 · 设计原则 (背下来能讲)

1. Orchestrator 发任务表 , Agent 按表执行
2. 计划内部存储 , 默认不展示
3. 阶段内可并行 , artifacts 下传
4. 每个 Phase 过 Harness
5. 三层路由 : Triage、Step Router、Search Router
6. Learner 永不自动改生产规则
7. 你定义「完成」 : milestone + 总反馈
8. 诚实能力边界 : 账号、支付、直发
9. Mock 测流程 , 真实 API 小批测质量
10. Builder 工具永不自动上线

Part 14 · 相关文档索引

- * docs/PAHS_深度学习教程_详细版.md (代码走读)
- * docs/PAHS_架构流程图.md
- * docs/ARCHITECTURE.md
- * docs/EXTERNAL_AGENTS_SETUP.md
- * docs/DEVLAB_ROADMAP.md
- * README.md (最新总览)

附录 · 术语中英对照

- 任务表 -> ExecutionPlan
- 阶段审核 -> Milestone review
- 总反馈 -> Final feedback
- 提案 -> Proposal
- 能力缺口 -> Capability gap
- 外部工具 -> External agent
- 批量测试 -> dev-batch

PAHS 学习全集 · Zachary Zhou · 2026-06